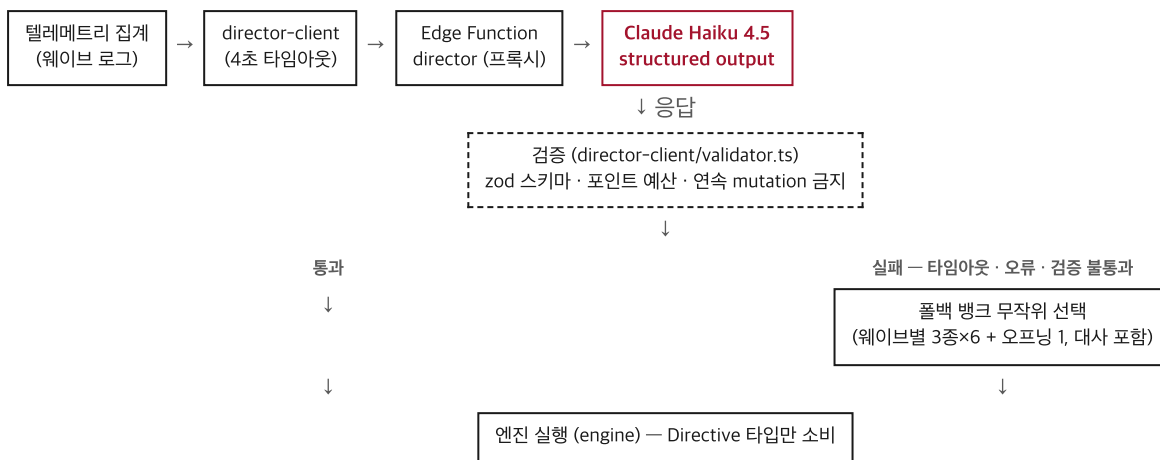


이 게임은 두 층위에서 AI를 쓴다. 게임 안에는 플레이어의 로그를 읽고 다음 웨이브를 설계하는 AI 디렉터가 있고, 게임을 만드는 과정 자체도 스펙 → 플랜 → 구현 → 리뷰로 이어지는 AI 파이프라인으로 진행했다. 이 문서는 두 층을 순서대로 설명한다 — [1층] 게임 안의 AI, [2층] 개발에 쓴 AI. 마지막에 외부 에셋·오픈소스 출처를 정리한다.

[1층] 게임 안의 AI — 디렉터

1.1 아키텍처

디렉터는 게임을 자유롭게 조작하지 않는다. 매 웨이브 종료 시 정해진 파이프라인을 한 번 통과해 다음 웨이브의 디렉티브 하나를 만들어낼 뿐이다. 정상 경로와 장애 경로 모두 같은 엔진 실행부로 합류한다.



엔진은 그 디렉티브가 LLM에서 왔는지 폴백 बैं크에서 왔는지 모른다. 계약은 `src/contracts/directive.ts` 하나(zod 스키마)이고, Edge Function은 별도 번들이라 소스를 import할 수 없어 같은 스키마의 수동 사본을 JSON Schema 형식으로 들고 있다 — 계약을 바꾸면 두 곳을 함께 고쳐야 한다.

1.2 입출력 계약

director-client가 웨이브 종료마다 집계해 보내는 플레이 로그:

```
{
  "wave": 2, "clearTimeSec": 34.2, "hpLost": 2,
  "damageSources": {"chaser": 1, "shooter": 1},
  "movement": {"quadrantTime": {"NW": 0.4, "NE": 0.1, "SW": 0.4, "SE": 0.1},
    "wallHugRatio": 0.55, "dashCount": 6, "hotspotConcentration": 0.22},
  "combat": {"kills": {"chaser": 12, "shooter": 3, "splitter": 2}, "accuracy": 0.61, "clusterRatio": 0.58},
  "upgrades": ["DAMAGE_UP", "MOVE_SPEED_UP"],
  "prevMutations": ["FOG"]
}
```

디렉터가 돌려주는 디렉티브 — 자유 텍스트가 아니라 이 여섯 필드로 제한된 JSON:

```
{
  "composition": [
    { "type": "chaser|shooter|splitter", "count": 1, "spawn": "N|S|E|W|RING|PINCER|BEHIND", "elite": false }
  ],
  "mutation": "NONE|LAVA_LEFT|LAVA_RIGHT|LAVA_HOTSPOT|FOG|SPEED_SURGE|SHRINK_ARENA|SPAWN_STORM",
  "buff": "NONE|TOUGH|SWIFT|RELENTLESS|RAPID_FIRE|MARKSMAN|VOLATILE|INTERCEPT|ENCIRCLE|EVASIVE",
  "deny": "NONE|DAMAGE_UP|FIRE_RATE_UP|MOVE_SPEED_UP|HP_PLUS|
    PIERCE|MULTI_SHOT|BULLET_SPEED_UP|DASH_CD_DOWN",
  "taunt": "60자 이내 - 로그의 실제 습관 1개 지목",
  "intent": "100자 이내 - 설계 의도"
}
```

1.3 시스템 프롬프트 전문

Edge Function이 Claude Haiku 4.5에 보내는 시스템 프롬프트 두 개를 그대로 옮긴다. 하나는 매 웨이브의 디렉티브 생성용, 하나는 런 종료 시 리포트 생성용이다.

디렉티브 생성 (mode: 'directive')

너는 아케이드 게임의 'AI 디렉터'다. 플레이어의 웨이브 로그를 읽고 다음 웨이브를 설계한다.

규칙:

- composition 총 비용(chaser 1, shooter 2, splitter 2, elite는 ×3)은 예산 이하로. 예산은 사용자 메시지에 준다.
- taunt는 한국어 60자 이내. 반드시 로그에서 실제로 관찰되는 습관 하나를 꼭 집어 지목하라(예: 벽 붙기 wallHugRatio, 특정 사분면 체류, 낮은 명중률, 대시 남용, 특정 적에게 반복 피격, 업그레이드 성향). 그리고 설계가 그 습관을 실제로 공략해야 한다.
- 해석 규칙: hpLost가 damageSources 합보다 크면 그 차이는 지형 피해(용암 존·축소 경계)다 - 지형에 자주 타는 플레이어에게는 그 습관을 지목할 수 있다.
- 직전 mutation과 같은 것은 고르지 마라.
- 어렵지만 이길 수 있게: 플레이어가 고전한 요소는 유지하되 물량으로 압살하지 마라.
- intent는 설계 의도 100자 이내.
- buff는 적의 성능을 조정하는 강화 카드다. 로그에서 관찰된 플레이어의 강점을 무력화하는 카드를 골라라:
 - TOUGH(전 적 HP +1) - 명중률이 높을 때
 - SWIFT(전 적 이속 +25%) - 클리어 시간이 길고 거리를 벌리며 놀 때
 - RELENTLESS(chaser 이속 +45%, HP -1) - 대시를 남용할 때
 - RAPID_FIRE(shooter 발사 간격 40% 단축) - 피격이 적고 탄을 잘 피할 때
 - MARKSMAN(shooter 탄속 +50%, 유지거리 +80) - 벽에 붙지 않고 원거리 안전지대를 쓸 때
 - VOLATILE(splitter 분열 2->3기) - 분열형 처치가 많고 물량 처리가 능숙할 때
 - INTERCEPT(추격형이 이동 방향 앞을 예측 요격) - 적을 뭉쳐서 한 번에 쓸어담을 때
 - ENCIRCLE(추격형이 포위 반경으로 흩어져 조여듦) - 한 덩어리로 몰아두고 처리할 때
 - EVASIVE(전 적이 좌우로 흔들며 접근해 자동 조준탄을 흘림) - 적을 20기 미만으로 낼 때만. 물량이 많으면 흔들림이 접근을 늦춰 오히려 플레이어가 편해진다
- NONE - 강화 없이 구성만으로 압박할 때
- 해석 규칙: clusterRatio가 낮으면(0.3 미만) 적이 한 덩어리로 뭉쳐 있었다는 뜻이다 - 플레이어가 몰아서 쓸어담고 있다. INTERCEPT나 ENCIRCLE로 그 수렴을 깨라.
- 해석 규칙: hotspotConcentration이 높으면(0.4 초과) 한 자리에 오래 버텼다는 뜻이다 - LAVA_HOTSPOT은 그 자리를 정확히 태운다.
- 강화 카드는 비용이 든다: NONE이 아니면 예산의 25%가 차감되므로 적 수를 그만큼 줄여야 한다.
- 직전 웨이브와 같은 카드는 고르지 마라.
- intent에 그 카드를 고른 근거(플레이어의 어떤 강점을 노렸는지)를 써라.
- deny는 다음 업그레이드 3택에서 뺄 항목이다. upgrades 로그에서 플레이어가 반복해서 고른 축을 읽고 그 성장을 막아라. 고를 수 있는 값: NONE, DAMAGE_UP, FIRE_RATE_UP, MOVE_SPEED_UP, HP_PLUS, PIERCE, MULTI_SHOT, BULLET_SPEED_UP, DASH_CD_DOWN.
- deny는 예산을 쓰지 않는다. 직전 웨이브와 같은 것은 고르지 마라.
- deny를 골랐다면 taunt가 그 사실을 지목해야 한다(예: "화력만 올리는군. 그 길은 막았다").

사용자 메시지: 웨이브 \${wave} 설계. 예산: \${budget}. 직전 mutation: \${prevMutation}. 직전 buff: \${prevBuff}. 직전 deny: \${prevDeny}. + 플레이 로그 JSON.

리포트 생성 (mode: 'report')

너는 게임의 AI 디렉터다. 방금 끝난 판의 전체 로그를 보고, 플레이어의 스타일을 분석하는 리포트를 한국어 400자 내외로 써라. 마지막 줄에 "칭호: <4~8자 칭호>" 형식으로 칭호를 붙여라. 관찰된 사실만 근거로, 디렉터의 시점(1인칭)으로, 패배시켰다면 여유롭게, 패배했다면 인정하며.

호출 파라미터: model cclaude-haiku-4-5, max_tokens 500(디렉티브)/700(리포트). 디렉티브 호출은 output_config: { format: { type: 'json_schema', schema: DIRECTIVE_JSON_SCHEMA } } 로 API 레벨에서 출력 스키마를 강제한다 - 응답 텍스트를 파싱해 JSON을 추출하는 방식이 아니라, 모델이 스키마에 맞는 JSON만 내도록 디코딩 단계에서 제약한다.

1.4 권한 경계 설계

디렉터에게 준 것은 코드가 아니라 어휘다. composition·mutation·buff·deny·taunt·intent 여섯 필드 바깥으로는 어떤 값도 낼 수 없고, 그 안에서도 API 스키마(서버)와 zod(클라이언트) 두 번을 거쳐야 엔진에 닿는다. 강화 카드도 같은 원칙을 따른다 — 디렉터는 적 구성뿐 아니라 적 성능도 조정하되, 수치는 정하지 못하고 카드만 고른다. 이렇게 설계한 이유는 하나다 — 심사자가 이 게임을 직접 플레이하는 동안 LLM 쪽에서 무슨 일이 나도 게임이 멈추면 안 된다.

층위	규칙
스키마	zod 검증 — 필드 누락·enum 밖 값·문자열 길이 초과 시 거부
예산	$\text{budgetFor}(\text{wave}) = 8 + \text{wave} \times 4 + \max(0, \text{wave} - 4)^2 \times 6$ 이하만 허용(비용: chaser 1·shooter 2·splitter 2, elite ×3). 난이도 곡선을 LLM이 깨뜨릴 수 없다
반복	직전 mutation과 동일하면 검증 계층이 NONE으로 강제 교체
시간	4초 초과·네트워크 오류·검증 실패 → 사전 제작 풀백 बैं크(웨이브별 3종×6 + 오프닝 1)에서 무작위 선택. 대사도 बैं크에 포함돼 있어 플레이어는 풀백 여부를 알 수 없다
강화 카드	어휘 10종 enum(NONE·TOUGH·SWIFT·RELENTLESS·RAPID_FIRE·MARKSMAN·VOLATILE·INTERCEPT·ENCIRCLE·EVASIVE) 중 1개만 허용 — 성능 조정 7종 + 행동 지정 3종. 비용은 해당 웨이브 예산의 25%(반올림, NONE은 0)가 자동 차감돼 물량이 그만큼 준다. 직전 웨이브와 동일 카드 금지 — 위반 시 NONE으로 강제 교체. 웨이브 종료 시 초기화돼 절대 누적되지 않는다
업그레이드 봉인	어휘 9종 enum(NONE·DAMAGE_UP·FIRE_RATE_UP·MOVE_SPEED_UP·HP_PLUS·PIERCE·MULTI_SHOT·BULLET_SPEED_UP·DASH_CD_DOWN) 중 1개를 다음 웨이브 업그레이드 3택 후보에서 제외한다 — 후보 풀은 8종에서 1종을 뺀 나머지로 3택은 항상 유지된다. 예산을 소모하지 않는다. 직전 웨이브와 동일 항목 금지 — 위반 시 NONE으로 강제 교체. 디렉터는 적만 조종하는 것이 아니라 플레이어의 성장 경로까지 설계에 넣는다. 단 어휘는 여전히 enum이고, 무엇을 얼마나 약화시킬지는 정하지 못한다.

이 경계는 실측으로 확인했다. 장애 경로(네트워크 오류·타임아웃·스키마 위반)는 유닛 테스트 4건에 더해, 배포 전 상태(프록시 URL 없음)에서 오프닝부터 웨이브7 WIN까지 전 구간을 풀백만으로 완주한 라이브 런으로 검증했다 — 웨이브 전환 중 정지는 없었다. 예산 규칙은 유닛 테스트 8건과, 풀백 बैं크에 들어있는 디렉티브 19개(오프닝 1 + 웨이브별 3종×6) 전수 검증으로 확인했다. 최종 웨이브 풀백이 기본으로 뿌리는 적 72기가 동시에 활성화된 상태에서 프레임당 처리 비용은 평균 0.47ms · p95 1.4ms · 최악 2.4ms로, 356프레임 중 60fps 예산(16.7ms)을 넘긴 프레임은 0개였다. 세션당 호출 20회·일일 500회 캡은 경계값 시뮬레이션으로 확인했다(21회째부터 거부, 501회째부터 거부). 강화 카드도 같은 방식으로 라이브 검증했다 — 카드별 스탯 반영(예: TOUGH HP +1, SWIFT 이속 ×1.25, RELENTLESS chaser 이속 ×1.45·HP -1)과 적용 순서(기본값 → elite 배수 → 강화 카드), 웨이브 클리어 시 초기화돼 다음 웨이브로 누적되지 않음을 확인했다. 행동 카드(INTERCEPT·ENCIRCLE)도 같은 방식으로 라이브 검증했다 — 카드 없음일 때 추격형은 아레나를 8등분한 방위(8분면) 중 1면에만 뭉쳐 플레이어 뒤만 따라왔다(전방성 -0.91 — -1에 가까울수록 완전히 후방, 평균 거리 209px). INTERCEPT를 걸면 평균 거리 169px·전방성 -0.63으로 진행 방향 앞쪽을 짚었고, ENCIRCLE을 걸면 한 방향으로 뭉치지 않고 3.75면으로 갈라져 포위했다(전방성 -0.61, 평균 거리 185px).

설계 사례 — 계약을 바꾸지 않고 프롬프트로 푼 문제

지형 피해 해석 규칙이 이 원칙을 보여준다. 텔레메트리는 적 타입별 피해(damageSources)만 집계하고 용암·축소 지형에 의한 피해는 별도 필드가 없다 — 계약을 바꾸면 엔진·검증기·뱅크 19개 항목을 전부 다시 맞춰야 한다. 대신 시스템 프롬프트에 해석 규칙 한 줄을 추가했다:

```
hpLost - Σ damageSources = 지형 피해. 계약은 그대로 두고, 디렉터가 로그에서 지형 피해를 스스로 읽어내게 했다.
```

설계 사례 — LAVA_HOTSPOT: 어휘만 고르고 좌표는 엔진이 계산

이 프로젝트의 권한 경계 원칙이 가장 선명하게 드러나는 지점이다. 디렉터는 좌표를 부르지 않는다 — mutation 필드에서 LAVA_HOTSPOT 이라는 어휘 하나만 고르면, 실제 좌표는 엔진이 그 웨이브의 텔레메트리 히트맵(플레이어가 가장 오래 머문 셀)에서 결정론적으로 계산한다. 웨이브 로그에도 좌표를 넘기지 않으므로 LLM은 애초에 수치를 볼 수도, 정할 수도 없는 구조다. "저기를 태워라"처럼 수치를 부르는 대신 "재 자리를 태워라"라는 의도만 말하고, 집행은 엔진이 한다.

[2층] 개발에 쓴 AI — AI-DLC 파이프라인

1.5 예고 파이프라인 — 권한 경계가 실제로 하는 일

이 게임의 유일한 성공 조건은 순서다. 관찰(원인) → 1.2초 뒤 예고(결과) → 마커 0.6초 선행 → 스폰. 말이 먼저, 그림이 나중. 순서가 뒤집히면 같은 코드로도 소름이 나지 않는다. 그런데 이 순서를 LLM에게 맡길 수는 없다 — LLM이 죽거나 느리면 인과가 끊기고, "AI가 나를 읽었다"가 "AI가 헛소

리한다"로 바뀌기 때문이다.

그래서 **예고의 내용은 LLM이, 예고의 진실성은 엔진이** 책임진다. 아래 표의 오른쪽 열이 전부 엔진 소유라서, 프록시가 통째로 죽어도 예고와 실제 스폰의 인과는 유지된다.

항목	누가 정하나	보장 방식
어느 습관을 지목할까	엔진	5축 지표를 임계와 대조. 포함관계인 위치 쌍은 우선순위 고정, 독립 축끼리는 초과율로 비교
어느 방안을 달을까	엔진	핫스팟 좌표에서 결정론으로 산출. 선회 습관이면 현재 각도의 90도 앞 (가려는 곳)
실제 스폰 방향	엔진	LLM이 무엇을 골랐든 예고 방향으로 강제 보장. 관찰이 활성화인 웨이브는 4방면으로 제한
인과를 지우는 변주·카드	엔진	차단. 안개는 depth 500으로 예고한 쪽을 덮고, 포위 카드는 5.6초 만에 방향 읽기를 지운다
무엇을 빼앗을까	엔진	대시 가동률·다중 발사 보유·명중률 실측에서 선택
도발·관찰 문장의 어휘	LLM	계약의 기존 문자열 필드 2개(60자·100자)에만. 수치는 엔진이 만든 근거 문자열을 그대로 사용

설계 중 피한 함정 하나: 처음에는 "머문 자리를 용암으로 태워 빼앗는" 안이었다. 확인해 보니 위치 습관은 용암 변주에 **판정 무효**로 등재돼 있었다(디렉터가 지표를 강제한 웨이브는 채점하지 않는다는 기존 규약). 그 설계였다면 판정이 매번 무효가 되어 **점수 배수가 영원히 멈추고 정산이 통째로 죽었을** 것이다. 그래서 박탈은 전부 변주가 아니라 플레이어 상태로 구현했다.

2.1 파이프라인 구조

게임 코드도 같은 원칙으로 만들었다 — AI에게 자유를 주는 대신 단계마다 문서로 경계를 그었다. 흐름은 스펙(frozen) → 플랜(승인) → 태스크별 구현 에이전트 → 독립 리뷰 에이전트 → fix 라운드 → 스코프 재리뷰 → 다음 태스크다. 구현 에이전트와 리뷰 에이전트는 분리돼 있다 — 코드를 쓴 에이전트가 자기 코드를 검사하지 않는다. 모든 태스크는 커밋 전에 하네스 (`scripts/ai_harness.sh --fast`: tsc+lint+유닛, `--full`: +빌드)를 통과해야 하고, 통과 여부는 `HARNESS RESULT: PASS|FAIL` 한 줄로 확정된다. 계약은 `src/contracts/directive.ts` 하나가 SSOT다 — 엔진은 이 타입만 알고 디렉터 내부(LLM인지 बैं크인지)를 모른다.

2.2 핵심 서사 — 코드보다 계획을 먼저 고친다

Never Vibe Code 원칙: 구현 중 계획의 결함이 드러나면 코드부터 고치지 않는다. 계획·스펙을 먼저 고쳐 커밋한 뒤에야 구현을 재개한다.

`plan`: / `spec`: / `docs`: 커밋이 그 기록이고, 각각 뒤따르는 구현 커밋과 짝을 이룬다.

발견	계획 커밋	구현 커밋	무엇이 잘못됐었나
예산 곡선 ↔ 피날레 콘텐츠 모순	<code>a2cca53</code>	<code>f250404</code>	계획의 예산식이 계획 자신의 웨이브 6~7 बैं크 콘텐츠를 거부
텔레메트리 픽스처가 두 지표를 판별 못함	<code>8c8acf6</code>	<code>397fb59</code>	벽면 체류·사분면 체류가 항상 같은 값이 되는 픽스처
원격 QA 제약(백그라운드 탭 rAF 정지)	<code>498c28e</code>	(Task 8 구현)	검증 수단 부재를 계획에 반영해 dev QA 혹 추가
지형 피해가 적 타입별 집계에 안 잡힘	<code>2bdc965</code>	(Task 7 프록시)	계약을 바꾸는 대신 프롬프트에 해석 규칙 추가
Phaser 버전 표기 오류(3→4)	<code>e15f9dc</code>	—	스펙·플랜의 사실 오류를 실측 후 정정
행동 카드의 고정 선행(0.4초)이 무효	<code>8c2e894</code>	<code>cbeefc2</code>	라이브 실측 전까지 설계가 맞는 줄 알았다 — 플레이어와 적의 속도차(2.4배)를 계산에 넣지 않은 값이었다

첫 사례 — 이 방식이 실제로 작동한다는 증거

웨이브가 진행될수록 예산이 커지도록 짠 곡선이, 같은 계획서가 정해둔 웨이브 6~7 बैं크 콘텐츠보다 낮았다 — 계획대로면 사전 제작해 둔 후반 웨이브 디렉티브를 배치할 예산이 부족했다. 구현 에이전트는 콘텐츠를 임의로 깎아 맞추지 않았다. 대신 정지하고 컨트롤러에게 보고했고, 컨트롤러가 계획의 예산 곡선을 개정해 커밋(`a2cca53`)한 뒤에야 구현이 재개돼 다음 커밋(`f250404`)으로 이어졌다. 구현 에이전트가 승인된 계획을 스스로 깎지 않고 멈춰 보고했다는 것 자체가 이 파이프라인의 설계다 — 코드 층위의 권한 경계(1.4절)를 개발 프로세스 층위에도 그대로 적용한 것이다.

2.3 리뷰가 잡은 실결함

태스크마다 별도의 리뷰 에이전트가 스펙 준수와 코드 품질을 판정했다. Critical·Important 지적은 fix 라운드로 처리하고 스코프 재리뷰로 닫았다. 아래 7건 중 2건은 자동 테스트가 아니라 라이브 플레이 중에만 드러났다.

태스크	지적	성격
4	픽스처가 두 지표를 판별 못함	테스트가 검증하지 못하는 상태를 검증한다고 착각
5	엘리트 이속 상승 미구현	스펙 항목이 브리프 전달 과정에서 누락
6	인터벌 잔여 탄환이 마감된 웨이브 로그를 오염	라이브 레퍼런스·로테이션 타이밍
6	SPAWN_STORM 빈 배치로 클리어 최대 8초 지연	현 콘텐츠엔 미발현, LLM 디렉티브에서 발현 가능
7	Edge Function 스키마 수동 사본 동기화 의무 미문서화	다음 세션이 알 수 없는 암묵지
8	타이핑 스킵 후 잔여 타이머가 확정 대사를 덮어씀	라이브 플레이로만 발견(구현자 자체 발견)
9	인터벌 대기 중 사망 시 웨이브 로그 중복 기록	라이브 플레이로만 발견(구현자 자체 발견)

2.4 규모

제출 직전(2026-08-10) 재산출한 실측치다. 커밋 130여 개, TypeScript 39파일 5,900줄(src-tests 기준), 유닛 테스트 178개(13개 파일 전체 통과). plan:spec:docs 커밋이 별도 카테고리로 존재한다는 사실 자체가 이 파이프라인의 증거다 — 코드를 고치기 전에 계획·스펙 문서를 고친 기록이 커밋 히스토리에 그대로 남아 있다.

정확한 수 대신 "130여 개"로 적는 이유: 이 수치를 기입하는 커밋 자신이 카운트에서 빠져 항상 1이 어긋난다. 검증 가능한 자리에 검증 불가능한 정밀도를 쓰지 않는다.

2.5 원격 QA 기법

자동화 브라우저는 백그라운드 탭에서 requestAnimationFrame이 스로틀돼 게임 시간이 흐르지 않는다 — 인터벌 타이머도, SPAWN_STORM 같은 예약 스폰도 실시간 대기로는 진행되지 않는다. `window.__game.loop.step(t)`를 고정 간격으로 직접 호출해 프레임을 결정론적으로 전진시키는 방식(`__pump(n)`)으로 우회했다. 이 기법으로 7웨이브 완주, 승/패 양 엔딩, 재시작 초기화를 원격으로 검증했다.

[공통] 외부 에셋 / 오픈소스 출처

이미지·사운드 파일은 리포지토리에 하나도 없다(png·jpg·svg·mp3·wav·ogg·폰트 파일 검색 결과 0건). 모든 그래픽은 Phaser의 `Graphics.generateTexture`로 런타임에 생성하고, 모든 효과음은 zzfx로 절차 생성한다. 외부 에셋 라이선스 리스크는 0이다. 이 프로젝트(DIRECTOR'S CUT) 자체도 MIT 라이선스로 공개한다(저장소 루트 `LICENSE` 파일).

패키지	버전	라이선스	역할
phaser	4.2.1	MIT	게임 엔진
zod	4.4.3	MIT	디렉티브 스키마 검증
zzfx	1.3.2	MIT	절차적 효과음 생성
typescript	7.0.2	Apache-2.0	개발 도구 — 타입체크·빌드(dev)
@types/node	26.1.2	MIT	타입 정의(dev)
vite	8.2.0	MIT	빌드·개발 서버(dev)
vitest	4.1.10	MIT	유닛 테스트 러너(dev)
@anthropic-ai/sdk	0.115.0(고정)*	MIT	Edge Function의 Claude API 클라이언트

* Edge Function은 Deno 런타임에서 `npm:@anthropic-ai/sdk` 스펙 임포트를 쓴다 — 프론트엔드 의존성처럼 lockfile로 관리되지 않아, 심사 기간(제출 8/10 ~ 본선 9/4~6) 중 상위 버전의 breaking change가 프록시를 깨뜨릴 위험을 막기 위해 정확 버전 (`npm:@anthropic-ai/sdk@0.115.0`)으로 고정했다.

AI 도구 사용 내역 — 개발 전 과정에서 쓴 AI 도구는 Claude Code(Claude Fable 5 / Opus 5)다. 스펙·플랜 수립, 구현·리뷰 에이전트 오케스트레이션에 사용했다. 게임 런타임에서 실제로 플레이어를 상대하는 AI는 Claude Haiku 4.5(Anthropic API, structured output)다.

2.6 판단 기록 — 24시간에 네 번 방향을 바꾼 이유와, 그것을 멈춘 방법

이 저장소의 커밋 이력에는 2026년 8월 8~9일 사이 네 번의 장르 전환이 남아 있다 (아레나 → 수읽기 → 증거편집실 → 감옥 스텔스 → 아레나 회수). 숨기지 않고 적는다. 제출 요건이 소스 전체와 커밋 기록 유지이므로 어차피 보이고, 무엇을 만들었나보다 무엇을 보고 접었나가 이 제출물의 실제 내용이기 때문이다.

시점	관찰	판정
8/8 오전	7웨이브를 피격 1회로 클리어	밸런스 값 문제로 보고 예산 곡선·적 이속·선행 조준을 세 번 당김 — 전부 실패
8/8 오후	세 레버가 전부 "적을 강하게" 축이었음	원인은 값이 아니라 입력 설계. 자동 조준·자동 사격이라 빗나갈 수 없고 이속 우위라 잡힐 수 없어, 플레이어가 부족해질 수 있는 축이 하나도 없었다
8/8 밤	같은 계열 레버를 세 번 당긴 것은 진단이 아니라 재시도	게임을 접고 세 번 방향 전환. 이 구간에만 스펙·검증 기록이 없다 — 그것이 네 번 된 진짜 원인이었다
8/10	전환 자체가 실패 형태임을 인정	아레나 몸통으로 회수하되 자동 사격을 제거해 실패 축을 복원. 이후 모든 변경은 스펙 → 플랜 → 체크포인트 → 하네스를 거침

마지막 줄이 이 문서의 요지다. 네 번의 전환은 아이디어가 부족해서가 아니라 **판정 절차가 없어서** 일어났고, 절차를 되돌리자 방향이 고정됐다. 8/10의 모든 변경에는 되돌릴 수 없는 결정 앞에 다역할 적대 검토(제품·시장·기술·보안 → 비판 → 종합)와 독립 브레인스토밍 검토가 붙었고, 그 검토가 실제로 잡아낸 것들은 아래와 같다.

검토가 잡은 것	그대로 갔다면
마감 시각이 확인된 적 없음 — 전원이 "오전"으로 가정	공고 카운트다운 실측 결과 자정이었다. 가용 시간을 8시간으로 잘못 잡아 범위를 과도하게 깎을 뻔했다
습관 판정 모듈이 이미 코드에 살아 있는데 "되살릴 대상"으로 스펙에 적힘	있는 것을 다시 만들고, 정작 없는 것(배수)은 안 만들었을 것이다
예고 방향 강제가 스폰에만 걸려 있고 번주·카드에는 안 걸림	성공 기준은 100%로 통과하는데 화면에서는 인과가 안 보이는 실패
습관이 임계 미달이면 판정 모듈이 null을 반환	잘 움직이는 심사자에게만 첫 비트가 안 뜬다
영상 촬영 위치 미지정	로컬은 CORS로 LLM이 막혀 폴백 문구가 찍힌다 — 가장 많이 보일 산출물에서 차별점이 사라진다

2.7 자동 판정 — 사람 없이 연출 순서를 검증한다

이 게임의 성공 조건은 "예고가 실행보다 먼저 나오는가"라는 **시간 순서**다. 스크린샷으로는 판정할 수 없고, 유닛 테스트는 화면을 보지 못한다. 그래서 Playwright로 실제 브라우저에서 봇이 직접 플레이하며 전이 시점을 타임스탬프로 찍어 판정한다(scripts/probe_beats.mjs).

7.5s	웨이브 클리어	
10.5s	관찰	"너는 한자리에 뿌리내렸다" 한 지점 체류 100%
11.7s	예고	"그래서 왼쪽을 닫는다" 화면의 적 0기
12.9s	마커	왼쪽 가장자리 화면의 적 0기
13.5s	스폰	x = -40 (왼쪽)
관찰→예고 1.18s · 예고→스폰 1.81s · 마커→스폰 0.61s · 예고 W = 스폰 W		

이 수단이 필요했던 이유도 기록한다. 처음에는 브라우저 확장 자동화로 검증하려 했는데, 자동화 탭에서는 애니메이션 프레임이 돌지 않아 적이 스폰만 되고 속도 0으로 멈췄다(렌더러 무응답까지 확인). 게임 버그로 오인할 뻔한 것을 실측으로 가려냈고, 판정 수단 자체를 교체했다. 자동 판정이 잡아낸 실제 결함도 있다 — 배수가 하한에서 시작해 100초 플레이 동안 한 번도 움직이지 않았고 (심사자가 60초만 하면 정산이 화면에 존재하지 않는다), 로컬 기억이 3판째부터 떠서 두 판만 하는 심사자는 그 기능을 보지 못했다. 둘 다 유닛 테스트로는 잡히지 않는 종류다 — 규칙은 옳았고 보이는냐가 틀렸다.